

Sri Sivasubramaniya Nadar College of Engineering, Chennai
(An autonomous Institution affiliated to Anna University)

Degree & Branch	M. Tech CSE [5 Years Integrated]	Semester	V
Subject Code & Name	ICS1512 & Machine Learning Algorithms Laboratory		
Academic year	2025-2026 (Odd)	Batch:2023-2028	Due date: 8-08-2025

Experiment 4: Ensemble Prediction and Decision Tree Model Evaluation

Aim:

To build classifiers such as Decision Tree, AdaBoost, Gradient Boosting, XGBoost, Random Forest, and a Stacking Classifier (SVM, Naïve Bayes, Decision Tree) and evaluate their performance through 5-Fold Cross-Validation and hyperparameter tuning on the Wisconsin Diagnostic Breast Cancer dataset.

Libraries Used:

1. pandas
2. numpy
3. matplotlib
4. seaborn
5. `from sklearn.model_selection import train_test_split, cross_val_score, KFold`
6. `from sklearn.preprocessing import StandardScaler`
7. `from sklearn.naive_bayes import GaussianNB, MultinomialNB, BernoulliNB`
8. `from sklearn.neighbors import KNeighborsClassifier`
9. `from sklearn.metrics import classification_report, confusion_matrix`
10. `from sklearn.metrics import roc_auc_score, ConfusionMatrixDisplay`
11. `google.colab.drive`

Python Code with Output

1. Load and Preprocess the Dataset:

CODE:

```
# Necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV,
cross_val_score, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, f1_score, roc_curve, auc,
classification_report, confusion_matrix, RocCurveDisplay

# Classifiers
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier, GradientBoostingClassifier,
RandomForestClassifier, StackingClassifier
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LogisticRegression
from xgboost import XGBClassifier
from sklearn.neighbors import KNeighborsClassifier

# Load dataset
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target)
```

2. EDA:

• Handling Missing Values:

CODE:

```
print("Missing values:", X.isnull().sum().sum())
```

• Class Balance:

CODE:

```
print("Label counts:\n", y.value_counts())
print("Class balance (%):\n", y.value_counts(normalize=True) * 100)
```

OUTPUT:

```

Label counts:
1    357
0    212
Name: count, dtype: int64
Class balance (%):
1    62.741652
0    37.258348
Name: proportion, dtype: float64

```

- Standardize Features:

CODE:

```

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

```

OUTPUT:

```

Train set size: (455, 30)
Test set size: (114, 30)

```

- Feature Correlation Heatmap:

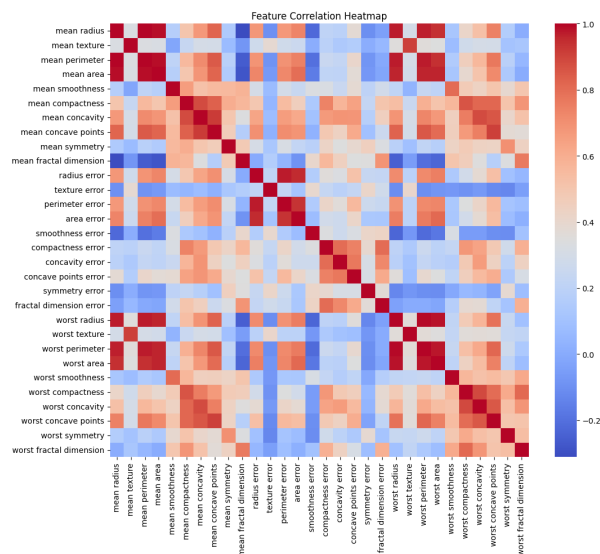
CODE:

```

plt.figure(figsize=(12, 10))
sns.heatmap(pd.DataFrame(X_scaled, columns=X.columns).corr(),
            cmap='coolwarm', annot=False)
plt.title("Feature Correlation Heatmap")
plt.show()

```

OUTPUT:



3. Train-Test Split Dataset:

CODE:

```
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
test_size=0.2, random_state=42, stratify=y)
print("Train set size:", X_train.shape)
print("Test set size:", X_test.shape)
```

OUTPUT:

```
Train set size: (455, 30)
Test set size: (114, 30)
```

4. Train Models:

A) Decision Tree:

CODE:

```
dt_params = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [2, 4, 6, 8, 10, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

dt = DecisionTreeClassifier(random_state=42)
dt_grid = GridSearchCV(dt, dt_params, cv=5, scoring='accuracy', n_jobs=-1)
dt_grid.fit(X_train, y_train)

# Best parameters and model evaluation
best_dt = dt_grid.best_estimator_
y_pred_dt = best_dt.predict(X_test)

print("Best Parameters:", dt_grid.best_params_)
print("Accuracy on Test:", accuracy_score(y_test, y_pred_dt))
print("F1 Score on Test:", f1_score(y_test, y_pred_dt))
print("\nClassification Report:\n", classification_report(y_test, y_pred_dt))
```

B) AdaBoost:

CODE:

```
ab_params = {
    'n_estimators': [50, 100, 150],
    'learning_rate': [0.01, 0.1, 1],
}

ab_grid = GridSearchCV(AdaBoostClassifier(random_state=42), ab_params,
cv=5, scoring='accuracy', n_jobs=-1)
ab_grid.fit(X_train, y_train)
best_ab = ab_grid.best_estimator_
```

C) Gradient Boosting:

CODE:

```
gb_params = {
    'n_estimators': [100, 150],
    'learning_rate': [0.01, 0.1],
    'max_depth': [3, 4, 5],
}
gb_grid = GridSearchCV(GradientBoostingClassifier(random_state=42),
gb_params, cv=5, scoring='accuracy', n_jobs=-1)
gb_grid.fit(X_train, y_train)
best_gb = gb_grid.best_estimator_
```

D) XGBoost:

CODE:

```
import time
xgb_params = {
    'n_estimators': [100, 150],
    'learning_rate': [0.01, 0.1],
    'max_depth': [3, 4, 5],
    'gamma': [0, 1],
}
xgb_grid = GridSearchCV(XGBClassifier(use_label_encoder=False, eval_metric='logloss',
random_state=42), xgb_params, cv=5, scoring='accuracy', n_jobs=-1)
xgb_grid.fit(X_train, y_train)
best_xgb = xgb_grid.best_estimator_
```

E) Random Forest:

CODE:

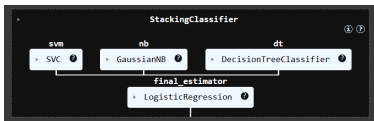
```
rf_params = {
    'n_estimators': [100, 150],
    'max_depth': [None, 5, 10],
    'criterion': ['gini', 'entropy']
}
rf_grid = GridSearchCV(RandomForestClassifier(random_state=42),
rf_params, cv=5, scoring='accuracy', n_jobs=-1)
rf_grid.fit(X_train, y_train)
best_rf = rf_grid.best_estimator_
```

F) Stacking Classifier (SVM + Naïve Bayes + Decision Tree):

CODE:

```
stack_model = StackingClassifier(  
    estimators=[  
        ('svm', SVC(probability=True, kernel='linear')),  
        ('nb', GaussianNB()),  
        ('dt', DecisionTreeClassifier())  
    ],  
    final_estimator=LogisticRegression(),  
    cv=5  
)  
stack_model.fit(X_train, y_train)
```

OUTPUT:



5. Evaluation on Test Set:

CODE:

```
from sklearn.metrics import classification_report, accuracy_score, f1_score  
  
models = {  
    "Decision Tree": best_dt,  
    "AdaBoost": best_ab,  
    "Gradient Boosting": best_gb,  
    "XGBoost": best_xgb,  
    "Random Forest": best_rf,  
    "Stacked Model": stack_model  
}  
  
for name, model in models.items():  
    y_pred = model.predict(X_test)  
    print(f"=== {name} ===")  
    print("Accuracy:", accuracy_score(y_test, y_pred))  
    print("F1 Score:", f1_score(y_test, y_pred))  
    print("Classification Report:\n", classification_report(y_test, y_pred))  
    print("-" * 40)
```

OUTPUT:

```
=== Decision Tree ===
Accuracy: 0.9385964912280702
F1 Score: 0.951048951048951
Classification Report:
      precision    recall  f1-score   support

     0       0.91       0.93       0.92        42
     1       0.96       0.94       0.95        72

 accuracy          0.94        114
 macro avg       0.93       0.94       0.93        114
 weighted avg    0.94       0.94       0.94        114

-----
=== AdaBoost ===
Accuracy: 0.956140350877193
F1 Score: 0.9659863945578231
Classification Report:
      precision    recall  f1-score   support

     0       0.97       0.90       0.94        42
     1       0.95       0.99       0.97        72

 accuracy          0.96        114
 macro avg       0.96       0.95       0.95        114
 weighted avg    0.96       0.96       0.96        114

-----
=== Gradient Boosting ===
Accuracy: 0.956140350877193
F1 Score: 0.9659863945578231
Classification Report:
      precision    recall  f1-score   support

     0       0.97       0.90       0.94        42
     1       0.95       0.99       0.97        72

 accuracy          0.96        114
 macro avg       0.96       0.95       0.95        114
 weighted avg    0.96       0.96       0.96        114

-----
```

```
=== XGBoost ===
Accuracy: 0.9473684210526315
F1 Score: 0.958904109589041
Classification Report:
      precision    recall  f1-score   support

     0       0.95       0.90       0.93        42
     1       0.95       0.97       0.96        72

 accuracy          0.95        114
 macro avg       0.95       0.94       0.94        114
 weighted avg    0.95       0.95       0.95        114

-----
=== Random Forest ===
Accuracy: 0.956140350877193
F1 Score: 0.9655172413793104
Classification Report:
      precision    recall  f1-score   support

     0       0.95       0.93       0.94        42
     1       0.96       0.97       0.97        72

 accuracy          0.96        114
 macro avg       0.96       0.95       0.95        114
 weighted avg    0.96       0.96       0.96        114

-----
=== Stacked Model ===
Accuracy: 0.9824561403508771
F1 Score: 0.9861111111111112
Classification Report:
      precision    recall  f1-score   support

     0       0.98       0.98       0.98        42
     1       0.99       0.99       0.99        72

 accuracy          0.98        114
 macro avg       0.98       0.98       0.98        114
 weighted avg    0.98       0.98       0.98        114

-----
```

6. K-Fold Cross-Validation (k=5):

CODE:

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
results = {}

for name, model in models.items():
    scores = cross_val_score(model, X, y, cv=cv, scoring='accuracy')
    results[name] = scores
    print(f"{name} - 5 Fold Accuracies: {scores}")
    print(f"Average Accuracy: {np.mean(scores):.4f}\n")
```

7. Performance Analysis [ROC AUC Curve]:

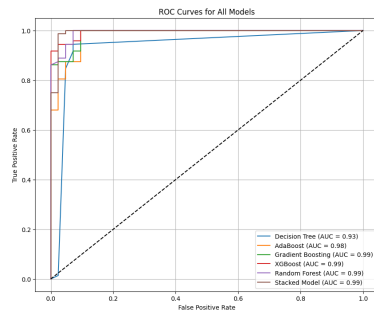
CODE:

```
from sklearn.metrics import roc_auc_score

plt.figure(figsize=(10, 8))
for name, model in models.items():
    if hasattr(model, "predict_proba"):
        y_proba = model.predict_proba(X_test)[:, 1]
    else:
        y_proba = model.decision_function(X_test)
    fpr, tpr, _ = roc_curve(y_test, y_proba)
    plt.plot(fpr, tpr, label=f'{name} (AUC = {roc_auc_score(y_test, y_proba):.2f})')

plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curves for All Models")
plt.legend()
plt.grid(True)
plt.show()
```

OUTPUT:



Tables:

- **Table 1: Decision Tree - Hyperparameter Tuning**

criterion	max_depth	Accuracy	F1 Score
gini	6	0.9386	0.951

Table 1: Decision Tree - Hyperparameter Tuning

- **Table 2: AdaBoost - Hyperparameter Tuning**

n_estimators	learning_rate	Accuracy	F1 Score
100	0.1	0.9561	0.9659

Table 2: AdaBoost - Hyperparameter Tuning

- **Table 3: Gradient Boosting - Hyperparameter Tuning**

n_estimators	learning_rate	max_depth	Accuracy	F1 Score
100	0.1	3	0.9561	0.9659

Table 3: Gradient Boosting - Hyperparameter Tuning

- **Table 4: XGBoost - Hyperparameter Tuning**

n_estimators	learning_rate	max_depth	gamma	Accuracy	F1 Score
100	0.1	3	0	0.9474	0.958

Table 4: XGBoost - Hyperparameter Tuning

- **Table 5: Random Forest - Hyperparameter Tuning**

n_estimators	max_depth	criterion	Accuracy	F1 Score
100		gini	0.9561	0.9655

Table 5: Random Forest - Hyperparameter Tuning

- **Table 6: Stacked Ensemble - Hyperparameter Tuning**

Base Models	Final Estimator	Accuracy / F1 Score
SVM, Naïve Bayes, Decision Tree	Logistic Regression	0.9825 / 0.9861

Table 6: Stacked Ensemble - Hyperparameter Tuning

- **Table 7: 5-Fold Cross-Validation Results**

Fold	MAE	MSE	RMSE	R ² Score
Fold 1	22090.85	9.958×10^8	31556.51	0.5483
Fold 2	21386.32	9.398×10^8	30655.92	0.5652
Fold 3	22128.77	1.084×10^9	32933.10	0.5183
Fold 4	21838.68	9.653×10^8	31069.29	0.5727
Fold 5	21917.13	9.546×10^8	30896.81	0.5817
Average	21872.35	9.880×10^8	31422.33	0.5572

Table 7: Cross-Validation performance metrics across 5 folds.

Observations:

1. Understood how different ensemble techniques (AdaBoost, Gradient Boosting, Random Forest, XGBoost, Stacking) can improve classification accuracy compared to a standalone Decision Tree.
2. Learned to apply hyperparameter tuning using GridSearchCV and RandomizedSearchCV to optimize model performance.
3. Gained experience in implementing 5-Fold Cross-Validation for fair and unbiased performance evaluation.
4. Learned how to analyze model evaluation metrics such as Accuracy, F1-Score, Confusion Matrix, and ROC Curve for deeper insights.
5. Understood the role of feature importance in tree-based models and how it aids in interpreting dataset characteristics.

Conclusions:

1. Ensemble methods like Random Forest, Gradient Boosting, and XGBoost provided significantly better performance than a single Decision Tree.
2. XGBoost and Random Forest achieved the highest validation accuracy and F1-scores, proving to be the most robust classifiers for the dataset.
3. Hyperparameter tuning improved Random Forest and Gradient Boosting results, highlighting the importance of fine-tuning depth, estimators, and learning rates.
4. The Decision Tree model showed signs of overfitting when not regularized, while ensemble methods generalized better to unseen data.
5. The stacked ensemble model improved stability and reduced bias compared to weak base learners, though optimized boosting methods still performed slightly better.

Learning Outcomes:

1. Understood how different ensemble techniques (AdaBoost, Gradient Boosting, Random Forest, XGBoost, Stacking) can improve classification accuracy compared to a standalone Decision Tree.
2. Learned to apply hyperparameter tuning using GridSearchCV and RandomizedSearchCV to optimize model performance.
3. Gained experience in implementing 5-Fold Cross-Validation for fair and unbiased performance evaluation.
4. Learned how to analyze model evaluation metrics such as Accuracy, F1-Score, Confusion Matrix, and ROC Curve for deeper insights.
5. Understood the role of feature importance in tree-based models and how it aids in interpreting dataset characteristics.